

PABV-STIN: A Proxy-Authorization-Based Batch Verification Protocol for Satellite-Terrestrial Integrated Network

Xin Qi¹, Caixia Liu^{2✉}, Yuzheng Liu³, Yulu Gao¹, and Yixuan Wu⁴

¹ Systems Engineering Institute, AMS, PLA, China
13002262871@163.com

higherthandeer@hrbeu.edu.cn

² Unit 32092 of PLA, China

lcxtxr@163.com

³ Harbin Engineering University, China

liuyuzheng@hrbeu.edu.cn

⁴ Northwestern Polytechnical University, China

wuyixuan1@mail.nwpu.edu.cn

Abstract. The short visibility windows of low Earth orbit (LEO) satellites and bursty access requests from massive Internet of Things (IoT) terminals pose major challenges to efficient and secure authentication in satellite-terrestrial integrated network (STIN). Existing authentication protocols often rely on real-time interactions with terrestrial entities, or they are mainly designed for single-user access and regular handover scenarios. These designs limit their applicability to large-scale concurrent terminal access within short LEO coverage windows. To address these limitations, this paper proposes PABV-STIN, a proxy-authorized batch verification protocol that supports mutual authentication and key agreement between users and LEO satellites. In PABV-STIN, a ground station (GS) issues time- and permission-constrained temporary proxy credentials to LEO satellites in advance. Authorized satellites can then authenticate users within the specified scope and validity period without frequent real-time satellite-ground interactions. PABV-STIN also aggregates concurrent authentication requests through a random-coefficient-based batch verification mechanism. When batch verification fails, abnormal requests are located through binary backtracking. The ROM-based formal proof, AVISPA-assisted verification, and heuristic security analysis indicate that PABV-STIN satisfies the security requirements defined for STIN access authentication under the considered threat model, including mutual authentication, session key agreement, proxy authorization security, and resistance to common attacks. Performance evaluation under 100-1000 concurrent users shows that PABV-STIN reduces authentication delay by up to 59.63% and communication overhead by up to 58.55% compared with representative related protocols. These results suggest that PABV-STIN is suitable for massive IoT terminal access in STIN.

Keywords: Satellite-terrestrial integrated network · Massive IoT access · Proxy-authorized authentication · Batch verification · Lightweight authentication.

1 Introduction

Satellite-terrestrial integrated network (STIN) connects satellites, terrestrial infrastructure, and heterogeneous terminals through inter-satellite and satellite-ground links. It provides essential network support for cross-domain communication [19, 34]. Owing to its wide-area coverage and flexible access capabilities, STIN is promising for emergency communication, disaster warning, and satellite Internet of Things (IoT) services [22, 18, 40]. For example, after a natural disaster, many distributed IoT terminals may need to access nearby low Earth orbit (LEO) satellites within a short visibility window to upload time-sensitive emergency sensing data. In such scenarios, secure and efficient access authentication is a prerequisite for establishing trusted communication relationships. However, satellite-ground links rely on open and long-range wireless channels. Authentication messages are therefore vulnerable to eavesdropping, replay, impersonation, and tampering attacks [39, 1]. Meanwhile, the short visibility windows and limited computing resources of LEO satellites make it challenging to process massive authentication requests within a limited time [28, 30, 35, 14]. Therefore, reducing authentication latency and communication overhead while maintaining authentication security has become an important challenge for large-scale concurrent access authentication in STIN [24].

Existing studies have proposed various authentication schemes for STIN [4, 7, 17, 36, 32, 23, 9, 13]. Nevertheless, many of them still face limitations in high-concurrency access scenarios under short LEO coverage windows. First, some schemes [4, 7, 17, 36, 23, 9] require the real-time participation of terrestrial entities, such as the network control center (NCC), during user access authentication. Although centralized participation facilitates credential management, it introduces additional satellite-ground interactions and increases authentication latency. Second, some schemes mainly focus on individual terminal access [4, 17, 23], roaming authentication [7] or handover authentication [36, 9], or pairwise authentication and key update between satellite-network entities [13]. In these schemes, each authentication instance is generally processed independently for an individual terminal or a pair of network entities, without aggregating concurrent terminal access requests into a single verification process. Such designs are difficult to scale to massive IoT terminals that initiate concentrated authentication requests within short LEO satellite coverage windows. Third, some schemes support batch verification. For example, the scheme in [32] employs a BVDC-based dichotomous localization mechanism to identify invalid requests when the initial batch verification fails. However, its batch verification relies on computationally expensive bilinear pairing operations, which must be repeatedly performed during rebatching, thereby increasing the authentication burden on resource-constrained LEO satellites.

In this work, we focus on enabling LEO satellites to authenticate a large number of IoT terminals within a limited contact duration while maintaining security and efficiency. Accordingly, our goal is to design a concurrent access authentication protocol for large-scale IoT access scenarios under the considered threat model. The protocol should reduce real-time satellite-ground interactions, lower authentication latency, and decrease communication overhead. To realize this goal, three key challenges must be addressed. The first challenge is how to delegate authentication capability from terrestrial infrastructure to LEO satellites without introducing excessive trust or unrestricted authorization. A naive solution is to allow satellites to authenticate users independently. However, this may weaken centralized control and increase the risk caused by compromised or misused credentials. The second challenge is how to support efficient batch authentication under constrained satellite resources. Repeatedly applying single-user authentication leads to high computational and communication overhead. Batch verification must also limit the risk of accepting forged or invalid requests hidden in a batch. The third challenge is how to identify abnormal requests when batch verification fails. Re-verifying all requests one by one may consume a substantial portion of the short satellite visibility window and weaken the efficiency advantage of batch authentication.

To address these challenges, this paper proposes PABV-STIN. It is a proxy-authorized batch authentication protocol for concurrent IoT terminal access. PABV-STIN uses a proxy authorization mechanism in which the ground station (GS) issues temporary proxy credentials with explicit validity periods and authorization scopes to legitimate LEO satellites before user access authentication. This design allows satellites to authenticate users within the authorized scope while preserving terrestrial-side control. PABV-STIN further constructs a Schnorr-type batch authentication mechanism to aggregate concurrent authentication requests and reduce repeated verification operations. Random small-exponent coefficients are introduced to reduce the probability that forged or invalid requests pass batch verification. When batch verification fails, PABV-STIN locates abnormal requests through binary backtracking. The main contributions of this paper are summarized as follows.

1. We design a proxy-authorized authentication mechanism with time and permission constraints for LEO satellite-assisted IoT access. By moving terrestrial authorization before user access authentication, the proposed mechanism enables LEO satellites to authenticate users within the authorized scope. This reduces real-time satellite-ground interactions and authentication latency.
2. We construct a Schnorr-type batch authentication mechanism for massive concurrent IoT access. The mechanism introduces random small-exponent coefficients to reduce the probability that forged or invalid requests pass batch verification. It also supports abnormal-request localization through binary backtracking when batch verification fails.
3. We provide AVISPA-assisted formal verification, ROM-based formal proof, and heuristic security analysis. The results indicate that PABV-STIN satis-

fies the security requirements defined for STIN access authentication under the considered threat model. These requirements include mutual authentication, session key agreement, proxy authorization security, and resistance to common attacks. We evaluate the performance of PABV-STIN under 100-1000 concurrent users and compare it with representative related protocols. Experimental results show that PABV-STIN reduces authentication delay by up to 59.63% and communication overhead by up to 58.55% compared with representative related protocols.

2 Related Work

Representative access authentication protocols for STIN can be grouped into three categories. The first category focuses on basic authentication and key agreement. Altaf et al. [2] and Soltani et al. [29] designed secure authentication and key agreement schemes for satellite communication systems relevant to STIN access authentication, improving mutual authentication and resistance to common attacks. However, these schemes mainly target regular single-user access. They do not fully address bursty access by massive terminals under short LEO coverage windows.

The second category considers mobility-oriented authentication or specific STIN authentication scenarios. Guo et al. [11] proposed an ECC-based access and handover authentication protocol. Guo et al. [8] further designed SRAKN to support secure roaming authentication with conditional anonymity and batch verification. Gautam et al. [6] proposed an ECC-based authentication protocol for satellite-to-satellite communication using satellite orbital coordinates. Xu et al. [37] proposed an energy-efficient access and handover scheme for 6G STIN through pre-negotiation and multiuser handover support. These schemes enhance authentication security in roaming, handover, or other STIN access scenarios. However, they are not primarily designed for high-concurrency initial access by massive IoT users within short LEO coverage windows. As a result, they may still incur high authentication latency and communication overhead when many terminals initiate access requests in a short period.

The third category introduces batch verification or multiuser authentication mechanisms to improve authentication efficiency under concurrent requests. Guo et al. [10] proposed PSEEMV, which applies ECC and the Chinese remainder theorem to delay-sensitive emergency message verification and supports batch verification. Guo et al. [12] proposed N3PA-STIN, a three-party authentication protocol for multiuser access in STIN. However, PSEEMV mainly targets emergency message verification rather than high-concurrency initial access authentication, while N3PA-STIN still involves additional interactions with terrestrial entities during high-concurrency access authentication. Moreover, when invalid requests are included in a batch, invalid-request localization after batch verification failure remains insufficiently addressed.

Therefore, designing a time- and permission-constrained proxy-authorized batch authentication mechanism remains an important problem for secure ac-

cess authentication in STIN. Such a mechanism should enable authorized LEO satellites to process massive concurrent authentication requests within a pre-authorized scope, reduce real-time satellite-ground interactions, and support invalid-request localization when batch verification fails.

3 System Model and Security Requirements

This section presents the system model, threat model, and security requirements for PABV-STIN. In addition, PABV-STIN relies on ECC, Schnorr-type signatures, and proxy signatures. Readers may refer to [25, 16, 26, 27, 15, 20] for detailed preliminaries.

3.1 System Model

This paper studies STIN for massive user access to LEO satellites. The system model includes the NCC, GS, LEO satellite, and users.

NCC. It is a fully trusted entity. It performs system initialization and entity registration, and issues registration and authentication parameters for GSs, LEO satellites, and users during registration. As the network management entity, it is equipped with strong security protection. This paper assumes that adversaries cannot obtain its master key or confidential data.

GS. It acts as the ground service entity and provides data transmission, remote monitoring, and emergency communication services for legitimate users. In the proposed protocol, it also implements proxy authorization. After authenticating an LEO satellite, it issues temporary proxy credentials with validity periods and authorization scopes.

LEO Satellite. It serves as the access gateway for users. With valid temporary proxy authorization from the GS, it receives and verifies anonymous access requests from users within its coverage area.

User. Users are service requesters, including smart terminals, vehicle-mounted devices, unmanned aerial vehicles, ships, and IoT sensor nodes.

3.2 Threat Model

This work adopts the Dolev–Yao model [5] and Canetti–Krawczyk model [3]. Authentication messages are transmitted over open satellite-terrestrial links, where adversaries can fully control public channels. They can monitor, intercept, delete, modify, reorder, and replay messages; impersonate legitimate users, LEO satellites, or GS; obtain one session key and attempt to derive other past or future session keys; and forge proxy delegations or abuse expired proxy credentials.

3.3 Security Requirements

Mutual Authentication. Mutual authentication shall be realized between users and authorized LEO satellites. GSs and LEO satellites shall also complete

mutual authentication before proxy delegation, so that proxy authorization is only granted to legitimate satellites.

Proxy Authorization Security. Proxy delegations issued by GSs shall have clear authorization scopes and validity periods. LEO satellites can only perform user access authentication within the scope and period specified by proxy credentials.

Unforgeability. Without legitimate private keys, proxy signing keys, and ephemeral random numbers, adversaries cannot forge user authentication requests, GS proxy delegations, or LEO satellite authentication responses.

Session Key Security and Forward Security. Users and LEO satellites shall negotiate secure session keys. Even if adversaries capture public messages, they cannot compute the session key. Even if one session key is compromised, adversaries cannot derive other past or future session keys.

Identity Privacy. Users' real identities shall not be exposed during access authentication. Adversaries cannot recover users' real identities from temporary identities or public authentication messages.

Batch Verification Security. LEO satellites shall support batch verification for multiple user authentication requests, and locate abnormal requests once batch verification fails.

Attack Resistance. The protocol shall resist common attacks over open satellite-terrestrial links, including replay attacks, tampering attacks, impersonation attacks, and MITM attacks, and other related attacks.

4 The Proposed Scheme

This section presents the detailed design of PABV-STIN. The protocol consists of five main phases: system initialization, entity registration, proxy authorization, user access authentication, and batch verification with abnormal request localization. For ease of reference, the main notations used in PABV-STIN are summarized in Appendix.

4.1 System Initialization

The NCC selects a security parameter λ , an elliptic-curve additive group G of prime order q with generator P , and a master key $x \in \mathbb{Z}_q^*$. It computes $P_{NCC} = xP$, defines secure hash functions $H(\cdot)$ and $h(\cdot)$, and publishes

$$Par = \{G, q, P, P_{NCC}, H, h, k\},$$

while keeping $msk = x$ secret.

4.2 Registration Phase

In the registration phase, the NCC generates registration and authentication parameters for the GS, LEO satellite, and users.

Infrastructure entity registration. Let $E_j \in \{G_s, L_e\}$ denote an infrastructure entity with identity ID_j . E_j submits ID_j to the NCC. The NCC randomly selects $r_j \in \mathbb{Z}_q^*$ and computes $K_j = r_j P, \sigma_j = xH(K_j, ID_j) + r_j \bmod q$. The NCC securely sends $\{K_j, \sigma_j\}$ to E_j .

The entity verifies $\sigma_j P \stackrel{?}{=} H(K_j, ID_j)P_{NCC} + K_j$. If the equation holds, E_j stores σ_j as its registration authentication credential. For $j = s$, E_j denotes the G_s ; for $j = e$, it denotes the L_e .

User registration. User U_i submits its real identity ID_{U_i} to the NCC. The NCC randomly selects $r_{U_i} \in \mathbb{Z}_q^*$ and computes $K_{U_i} = r_{U_i} P$. It generates the temporary identity $TID_{U_i} = h(K_{U_i}, x) \oplus ID_{U_i}$, issues the user credential w_{U_i} containing the user's access attributes, and computes $\sigma_{U_i} = xH(K_{U_i}, TID_{U_i}, w_{U_i}) + r_{U_i} \bmod q$. The NCC sends $\{K_{U_i}, \sigma_{U_i}, TID_{U_i}, w_{U_i}\}$ to U_i through a secure channel. Then, U_i verifies whether

$$\sigma_{U_i} P \stackrel{?}{=} H(K_{U_i}, TID_{U_i}, w_{U_i})P_{NCC} + K_{U_i}$$

holds. If the equation holds, U_i stores the corresponding registration authentication parameters.

4.3 Proxy Authorization Phase

Before user access authentication is performed, G_s and L_e first complete mutual authentication. Then, G_s generates temporary proxy credentials for the LEO satellite. The detailed process is as follows.

- **G_s authentication.** G_s randomly selects $n_s \in \mathbb{Z}_q^*$, computes $N_s = n_s P$, generates the timestamp ts_s , and constructs $M_s = ID_s \parallel K_s \parallel N_s \parallel ID_e \parallel ts_s$. Then, G_s generates the authentication signature $S_s = \sigma_s - H(M_s)n_s$ using its registration private key σ_s . G_s sends $\{ID_s, K_s, N_s, ts_s, S_s\}$ to L_e . After receiving the message, L_e checks the freshness of ts_s and verifies whether

$$S_s P + H(M_s)N_s \stackrel{?}{=} H(K_s, ID_s)P_{NCC} + K_s$$

holds. If the equation holds, L_e confirms that G_s is a legitimate GS.

- **L_e authentication.** Similarly, L_e selects $n_e \in \mathbb{Z}_q^*$, computes $N_e = n_e P$, generates ts_e , and forms $M_e = ID_e \parallel K_e \parallel N_e \parallel ID_s \parallel N_s \parallel ts_e$, $S_e = \sigma_e - H(M_e)n_e \bmod q$. It sends $\{ID_e, K_e, N_e, N_s, ts_e, S_e\}$ to the GS. After validating ts_e and the binding of N_s to the current session, the GS verifies

$$S_e P + H(M_e)N_e \stackrel{?}{=} H(K_e, ID_e)P_{NCC} + K_e.$$

If the equation holds, the GS authenticates L_e .

- **Session key generation.** The two entities derive the common session key

$$SK_{G_s-L_e} = SK_{L_e-G_s} = h(n_s N_e \parallel N_s \parallel N_e \parallel ID_s \parallel ID_e \parallel ts_s \parallel ts_e).$$

Since $n_s N_e = n_e N_s$, both parties obtain the same session key.

- **Proxy delegation generation and verification.** After mutual authentication, G_s constructs the proxy warrant w_{se} , which contains the identity of L_e , authorization scope and validity period. Then, G_s randomly selects $r_{se} \in Z_q^*$, computes $K_{se} = r_{se}P$, and generates the temporary proxy signing key $\sigma_{se} = \sigma_s H(K_{se}, ID_s, ID_e, w_{se}) + r_{se}$. G_s constructs the proxy delegation message $M_{se} = ID_s \parallel ID_e \parallel K_s \parallel K_{se} \parallel \sigma_{se} \parallel w_{se}$. To ensure confidentiality and integrity, G_s encrypts M_{se} as $C_{se} = Enc_{SK_{G_s-L_e}}(M_{se})$ and generates $Tag_{se} = MAC_{SK_{G_s-L_e}}(C_{se} \parallel ts_{se})$. Then, G_s sends $\{C_{se}, Tag_{se}, ts_{se}\}$ to L_e . L_e checks the validity of ts_{se} and verifies whether

$$Tag_{se} \stackrel{?}{=} MAC_{SK_{L_e-G_s}}(C_{se} \parallel ts_{se})$$

holds. If the verification succeeds, L_e decrypts C_{se} and obtains M_{se} . Then, it checks whether the validity period, authorization scope, and satellite identity in w_{se} are valid. Finally, L_e verifies whether

$$\sigma_{se}P \stackrel{?}{=} H(K_{se}, ID_s, ID_e, w_{se})[H(K_s, ID_s)P_{NCC} + K_s] + K_{se}$$

holds. If the equation holds, the proxy delegation is generated by a legitimate G_s . Then, L_e stores $\{\sigma_{se}, K_{se}, w_{se}, ID_s, K_s\}$ and uses σ_{se} to generate proxy authentication responses in later user access authentication.

4.4 User Access Authentication Phase

When user U_i needs to access the network, it initiates an anonymous access authentication request to L_e within the LEO satellite coverage window. The detailed process is as follows.

- **User access request generation.** U_i randomly selects $b_i \in Z_q^*$ and computes $R_{U_i} = b_iP$. Then, it generates timestamp ts_1 and constructs the authentication message $M_{U_i} = R_{U_i} \parallel ts_1 \parallel TID_{U_i} \parallel K_{U_i} \parallel w_{U_i}$. U_i generates the authentication signature $s_{U_i} = \sigma_{U_i} - H(M_{U_i})b_i$ using its registration private key σ_{U_i} . Then, U_i sends $\{M_{U_i}, s_{U_i}\}$ to L_e .
- **User request verification and satellite response generation.** After receiving the access request, L_e first checks the freshness of ts_1 and then verifies whether

$$s_{U_i}P + H(M_{U_i})R_{U_i} \stackrel{?}{=} H(K_{U_i}, TID_{U_i}, w_{U_i})P_{NCC} + K_{U_i}$$

holds. If the equation holds, the request is generated by a legitimate registered U_i . Then, L_e randomly selects $c_{Le} \in Z_q^*$, computes $R_{Le} = c_{Le}P$, generates timestamp ts_2 , and constructs the response message $M_{Le} = R_{Le} \parallel R_{U_i} \parallel TID_{U_i} \parallel ts_1 \parallel ts_2 \parallel ID_s \parallel ID_e \parallel K_s \parallel K_{se} \parallel w_{se}$. L_e generates the proxy response signature $s_{Le} = \sigma_{se} - H(M_{Le})c_{Le}$. Meanwhile, L_e computes the session key as $SK_{L_e-U_i} = h(c_{Le}R_{U_i} \parallel R_{U_i} \parallel R_{Le} \parallel TID_{U_i} \parallel ID_e \parallel ts_1 \parallel ts_2)$. Then, L_e sends M_{Le}, s_{Le} to the user.

- **Satellite response verification and session key generation.** After receiving the response, U_i checks the validity of w_{se} and ts_2 , and verifies whether

$$s_{Le}P + H(M_{Le})R_{Le} \stackrel{?}{=} H(K_{se}, ID_s, ID_e, w_{se})[H(K_s, ID_s)P_{NCC} + K_s] + K_{se}$$

holds. If the equation holds, U_i confirms that the response is generated by a legitimate L_e authorized by G_s . Then, U_i computes the session key as $SK_{U_i-L_e} = h(b_i R_{Le} \parallel R_{U_i} \parallel R_{Le} \parallel TID_{U_i} \parallel ID_e \parallel ts_1 \parallel ts_2)$. Since $b_i R_{Le} = b_i c_{Le} P = c_{Le} R_{U_i}$, both parties obtain the same session key.

4.5 Batch Verification and Abnormal Request Localization

In massive access scenarios, multiple users may initiate authentication requests within the same LEO satellite coverage window. If L_e verifies each request separately, the computational overhead will be high. Therefore, PABV-STIN uses random-coefficient batch verification to aggregate multiple user authentication requests. The batch access authentication and session key agreement phase is shown in Fig. 1. The detailed process is as follows.

- **Batch request verification.** Assume that L_e receives N users authentication requests $\{(M_{U_i}, s_{U_i})\}_{i=1}^N$ within the same time window. For each U_i request, L_e selects an independent random coefficient $\alpha_i \in \{1, \dots, 2^k - 1\}$. Then, L_e verifies the following batch authentication equation:

$$\begin{aligned} & \sum_{i=1}^N \alpha_i s_{U_i} P + \sum_{i=1}^N \alpha_i H(M_{U_i}) R_{U_i} \\ & \stackrel{?}{=} \sum_{i=1}^N \alpha_i H(K_{U_i}, TID_{U_i}, w_{U_i}) P_{NCC} + \sum_{i=1}^N \alpha_i K_{U_i}. \end{aligned}$$

If the equation holds, the batch of U_i requests is accepted.

- **Batch response generation.** After successful batch verification, L_e randomly selects $c_{Le} \in Z_q^*$, computes $R_{Le} = c_{Le} P$, generates timestamp ts_2 , and constructs $List = \{TID_{U_i}, R_{U_i}, ts_1\}_{i=1}^N$. Then, L_e constructs the batch response message $M_{Le}^{batch} = R_{Le} \parallel H(List) \parallel ts_2 \parallel ID_s \parallel ID_e \parallel K_s \parallel K_{se} \parallel w_{se}$. L_e generates the batch response signature $s_{Le}^{batch} = \sigma_{se} - H(M_{Le}^{batch})c_{Le}$ using the proxy signing key. Meanwhile, for each accepted user U_i , L_e computes the session key as $SK_{L_e-U_i} = h(c_{Le} R_{U_i} \parallel R_{U_i} \parallel R_{Le} \parallel TID_{U_i} \parallel ID_e \parallel ts_1 \parallel ts_2)$. Then, L_e broadcasts $\{List, M_{Le}^{batch}, s_{Le}^{batch}\}$ to all accepted U_i .
- **Batch response verification.** After receiving the batch response, each U_i first checks whether $(TID_{U_i}, R_{U_i}, ts_1)$ is included in $List$ and whether the value $H(List)$ in M_{Le}^{batch} is correct. Then, each U_i verifies whether

$$\begin{aligned} & s_{Le}^{batch} P + H(M_{Le}^{batch}) R_{Le} \\ & \stackrel{?}{=} H(K_{se}, ID_s, ID_e, w_{se})[H(K_s, ID_s)P_{NCC} + K_s] + K_{se} \end{aligned}$$

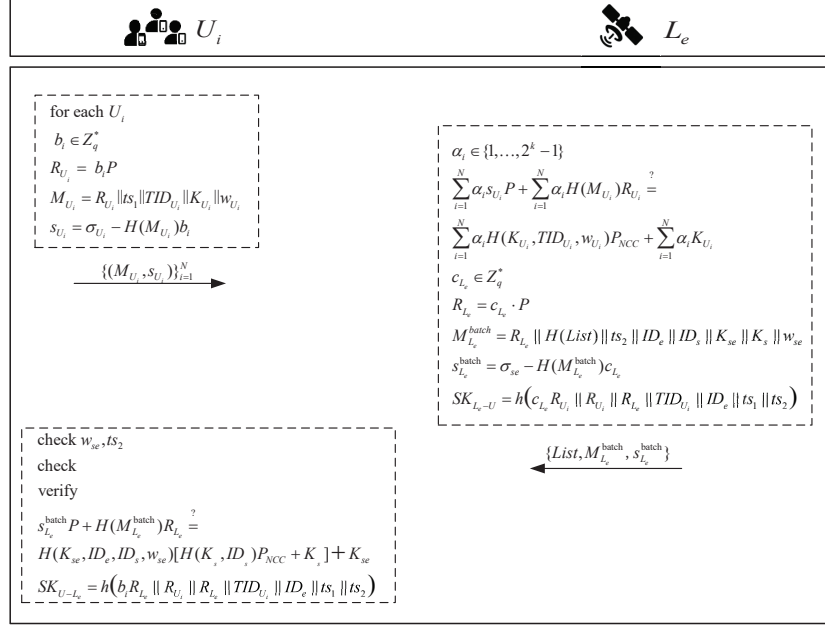


Fig. 1. Batch access authentication and session key agreement phase.

holds. If the equation holds, U_i accepts the batch response and computes the session key as $SK_{U_i, L_e} = h(b_i R_{L_e} \parallel R_{U_i} \parallel R_{L_e} \parallel TID_{U_i} \parallel ID_e \parallel ts_1 \parallel ts_2)$. Since $b_i R_{L_e} = b_i c_{L_e} P = c_{L_e} R_{U_i}$, U_i and L_e obtain the same session key.

- **Abnormal Request Localization.** If batch verification fails, L_e locates abnormal requests by binary backtracking. Define the weighted verification residual of request U_i as

$$Q_i = \alpha_i s_{U_i} P + \alpha_i H(M_{U_i}) R_{U_i} - \alpha_i H(K_{U_i}, TID_{U_i}, w_{U_i}) P_{NCC} - \alpha_i K_{U_i}.$$

During the initial batch verification, L_e constructs and stores the prefix aggregates $S_0 = \mathcal{O}$ and $S_j = \sum_{i=1}^j Q_i$, where $\mathcal{O}$ is the identity point. For any sub-batch $[a, b]$, L_e checks $S_b - S_{a-1} \stackrel{?}{=} \mathcal{O}$. If the sub-batch fails verification, it is further divided into two smaller sub-batches; otherwise, all requests in it are accepted. This process is repeated until the abnormal request is located. Thus, backtracking reuses the initial verification results without repeating scalar multiplications or hash computations.

5 Security Analysis

This section analyzes the security of PABV-STIN from three aspects. AVISPA is used to verify the authentication and secrecy goals, the ROM proof establishes the semantic security of the session key, and heuristic analysis shows that the protocol satisfies the required security properties and resists common attacks.

```

role user(Ui,Le,Gs: agent,
  Ks,Ke,Kui,TidUi,Wui,Dhul,P: text,
  SigUi: text,
  PKse: public_key,
  SND,RCV: channel(dy),
  H1,H2: hash_func)
played_by Ui
def=local State: nat,
  Bi,Rui,Rle,Ts1,Ts2,Kse,Wse,SKul: text
init State := 0
transition
1. State = 0 ∧ RCV(start) =>
  State' := 1
  ∧ Bi' := new()
  ∧ Rui' := H1(Bi'.P)
  ∧ Ts1' := new()
  ∧ SND(Rui'.Ts1'.TidUi.Kui.Wui.H1(Rui'.Ts1'.TidUi.Kui.Wui.SigUi))
  ∧ witness(Ui,Le,ui_le_auth,Rui')
  ∧ witness(Ui,Le,ui_le_ts1,Ts1')
2. State = 1
  ∧ RCV(Rle'.Rui.TidUi.Ts1.Ts2'.Le.Gs.Kse'.Ks.Wse'.{Rle'.Rui.TidUi.Ts1.Ts2'.Le.Gs.Kse'.Ks.Wse'}_inv(PKse))=>
  State' := 2
  ∧ SKul' := H2(Dhul.Rui.Rle'.TidUi.Le.Ts1.Ts2')
  ∧ secret(SKul',sk_ui_le,{Ui,Le})
  ∧ request(Ui,Le,le_ui_auth,Rle')
  ∧ request(Ui,Le,le_ui_ts2,Ts2')
end role

```

Fig. 2. HLPSTL role specification of U_i .

5.1 AVISPA Verification

We use HLPSTL to model U_i , L_e , G_s , session, and environment roles. The AVISPA tool is employed with the OFMC and CL-AtSe backends to verify the authentication and secrecy goals of PABV-STIN. Due to space limitations, only the user role is shown as an illustrative example. As shown in Figs. 2 and 3, the representative HLPSTL user role, the specified security goals, and the corresponding verification results are presented. Both OFMC and CL-AtSe return SAFE, indicating that the specified authentication and secrecy goals are satisfied. The complete HLPSTL model is provided in Appendix.

5.2 Formal Security Proof in ROM

This subsection proves the semantic security of the session key in the single-user authentication phase of PABV-STIN under the random oracle model (ROM). Let $\mathcal{A}$ be a probabilistic polynomial-time adversary interacting with instances Π_U^x and Π_L^y of U_i and L_e . The adversary is allowed to issue the following queries: **Execute** for obtaining passive transcripts, **Send** for active message injection, **Reveal** for obtaining accepted session keys, **LongTermKeyReveal** for obtaining long-term credentials of non-fresh sessions, random-oracle queries to H and h , and **Test** for distinguishing the real session key of a fresh instance from a random string. The advantage of $\mathcal{A}$ is defined as $Adv_{\mathcal{A}}^P = |2 \Pr[Succ] - 1|$.

<pre> goal secrecy_of sk_gs_le secrecy_of sk_ui_le authentication_on gs_le_auth authentication_on le_gs_auth authentication_on gs_le_proxy authentication_on ui_le_auth authentication_on ui_le_ts1 authentication_on le_ui_auth authentication_on le_ui_ts2 end goal </pre>	<pre> % OFMC % Version of 2006/02/13 SUMMARY SAFE DETAILS BOUNDED_NUMBER_OF_SESSIONS PROTOCOL /home/span/span/testsuite/results/PABV_STIN.if GOAL as_specified BACKEND OFMC COMMENTS STATISTICS parseTime: 0.00s searchTime: 0.00s visitedNodes: 10 nodes depth: 5 plies </pre>	<pre> SUMMARY SAFE DETAILS BOUNDED_NUMBER_OF_SESSIONS TYPED_MODEL PROTOCOL /home/span/span/testsuite/results/PABV_STIN.if GOAL As Specified BACKEND CL-AtSe STATISTICS Analysed : 8 states Reachable : 4 states Translation: 0.00 seconds Computation: 0.00 seconds </pre>
(a) Security goals	(b) OFMC result	(c) CL-AtSe result

Fig. 3. Security goals and AVISPA verification results of PABV-STIN.

Theorem 1. Assume that H and h are random oracles and that the Computational DiffieHellman (CDH) problem is hard in G . Let q_e , q_s , q_H , and q_h denote the maximum numbers of **Execute**, **Send**, H , and h queries, respectively. Let l_H and l_h be the output lengths of H and h . Then the advantage of $\mathcal{A}$ in breaking the session key security of PABV-STIN is bounded by $Adv_{\mathcal{A}}^P \leq \frac{q_H^2 + q_H q_s}{2^{l_H}} + \frac{q_h^2 + q_h q_s}{2^{l_h}} + \frac{(q_s + q_e)^2}{2q} + 2Adv_{\mathcal{A}}^{CDH}$.

Proof. The proof proceeds through a sequence of games. $Game_0$ is the real attack game, where $Adv_{\mathcal{A}}^P = |2\Pr[Succ_0] - 1|$. In $Game_1$, $\mathcal{A}$ obtains all public transcripts through **Execute** queries, including (M_{U_i}, s_{U_i}) and (M_{L_e}, s_{L_e}) . Since the ephemeral secrets b_i and c_{L_e} are not transmitted, this does not change the winning probability. In $Game_2$, collisions in H , h , and ephemeral random values are excluded. By the birthday bound, the probability of such collisions is bounded by the first three terms in the theorem. In $Game_3$, successful forgeries through **Send** queries are considered. Forging a valid user request or satellite response requires producing a valid Schnorr-type signature without the corresponding signing secret and ephemeral randomness. Such events are bounded by the random-oracle guessing terms included in the first two terms. In $Game_4$, long-term key exposure is considered under the freshness definition. For a fresh test session, the adversary cannot reveal long-term credentials in a way that violates freshness, and post-session exposure does not reveal the ephemeral DH value. In $Game_5$, the real shared value $b_i R_{L_e} = c_{L_e} R_{U_i} = b_i c_{L_e} P$ is replaced with a random group element. Any adversary that distinguishes this replacement can be used to solve the CDH problem in G . In $Game_6$, the input to h is independent of $\mathcal{A}$'s view, and hence the test key is indistinguishable from a random string.

Combining the above games with the triangle inequality gives the stated bound. Since the random-oracle output lengths are sufficiently large and the CDH problem is computationally hard in G , the above bound is negligible.

Therefore, the session key generated in the single-user authentication phase of PABV-STIN is semantically secure in the ROM.

5.3 Heuristic Security Analysis

Mutual authentication. PABV-STIN achieves mutual authentication at two levels. Before proxy delegation, G_s and L_e verify each other through the NCC-issued registration credentials. The validity of S_s and S_e proves that the proxy delegation is established only between a legitimate G_s and a legitimate L_e . During user access authentication, L_e verifies U_i request by checking $s_{U_i}P + H(M_{U_i})R_{U_i} \stackrel{?}{=} H(K_{U_i}, TID_{U_i}, w_{U_i})P_{NCC} + K_{U_i}$. Conversely, U_i verifies whether L_e response is generated under a valid proxy credential issued by G_s . Therefore, PABV-STIN achieves mutual authentication between U_i and the authorized L_e .

Proxy authorization security. After mutual authentication, G_s issues a proxy warrant w_{se} containing the satellite identity, authorization scope, validity period, and batch authentication permission. The temporary proxy signing key $\sigma_{se} = \sigma_s H(K_{se}, ID_s, ID_e, w_{se}) + r_{se}$ is bound to ID_s , ID_e , w_{se} , and K_{se} , and is transmitted under the protection of the session key shared by G_s and L_e . Thus, an adversary cannot transfer the proxy credential to another satellite, use it beyond the authorized scope, or forge a valid proxy delegation without knowing σ_s or r_{se} .

Unforgeability. For a user request, a valid signature has the form $s_{U_i} = \sigma_{U_i} - H(M_{U_i})b_i$. An adversary observing public messages cannot obtain σ_{U_i} or derive b_i from $R_{U_i} = b_iP$ due to the hardness of ECDLP. Similarly, forging a valid satellite response requires the temporary proxy signing key σ_{se} or a forged proxy delegation from G_s . Since these values are protected and bound to the corresponding identities and proxy warrant, PABV-STIN prevents the forgery of both user requests and satellite responses.

Session key security and forward security. The semantic security of the session key is formally proved in Section 5.2. For forward security, each session key is derived from fresh ephemeral secrets b_i and c_{L_e} , satisfying $b_i R_{L_e} = c_{L_e} R_{U_i} = b_i c_{L_e} P$. Since these secrets are independently generated for each session and are never transmitted, the compromise of one session key or a long-term credential after session completion does not reveal other past or future session keys.

Identity privacy. The real identity ID_{U_i} is not transmitted in plaintext during access authentication. Instead, U_i uses the temporary identity $TID_{U_i} = h(K_{U_i}, x) \oplus ID_{U_i}$. Since the NCC master key x is not available to external adversaries, ID_{U_i} cannot be recovered from TID_{U_i} or public authentication messages. Thus, PABV-STIN protects user identity privacy.

Batch verification security. For each valid user request, the individual verification equation holds. By multiplying each equation with an independent random coefficient α_i and aggregating all equations, L_e verifies the batch equation $\sum_{i=1}^N \alpha_i s_{U_i} P + \sum_{i=1}^N \alpha_i H(M_{U_i}) R_{U_i} \stackrel{?}{=} \sum_{i=1}^N \alpha_i H(K_{U_i}, TID_{U_i}, w_{U_i}) P_{NCC} + \sum_{i=1}^N \alpha_i K_{U_i}$. If all requests are valid, the equation holds. If invalid requests are

included, they pass the batch verification only with negligible probability due to the independently selected random coefficients. Therefore, the proposed batch verification mechanism preserves correctness and security.

Resistance to common attacks. PABV-STIN resists replay, impersonation, MITM, and tampering attacks. Fresh timestamps ts_s , ts_e , ts_{se} , ts_1 , and ts_2 prevent replayed messages from being accepted. Impersonation requires valid registration credentials, ephemeral secrets, or the temporary proxy signing key, which cannot be derived from public messages. Moreover, all critical messages are bound to identities, timestamps, random values, and credentials through hash-based signatures or MACs. Any modification changes the verification input and causes authentication failure.

6 Performance Evaluation

In this section, we evaluate the performance of the proposed PABV-STIN protocol and compare it with related protocols [38, 31, 11, 21, 33] in terms of signaling overhead, authentication delay, and communication overhead. We further analyze the backtracking overhead of PABV-STIN under batch verification failures.

6.1 Signaling Overhead Comparison

The signaling overhead refers to the number of authentication message transmissions required to complete the access authentication phase. The signaling overheads of protocols [38, 31, 11, 21, 33] and the proposed PABV-STIN protocol are 3, 3, 4, 2, 4, and 2, respectively. The signaling overhead of the proposed PABV-STIN protocol is the lowest, requiring only 2 message transmissions. Compared with protocols [38, 31, 11], and [33], the PABV-STIN protocol reduces the signaling overhead by 1, 1, 2, and 2 transmissions, respectively.

6.2 Authentication Delay Comparison

Authentication delay refers to the sum of the computational delay and authentication message transmission delay.

Table 1 reports the average execution time of each cryptographic operation measured over 1000 Charm runs on an Intel i9-14900HX CPU with 32 GB RAM. For a fair comparison, all schemes are evaluated using the same measured cryptographic operation times under the same experimental environment. Therefore, the reported delays mainly reflect the relative computational overhead of different schemes rather than the exact processing latency on real satellite platforms. We set $T_{s-g} = 10$ ms and $T_{g-g} = 5$ ms. For PABV-STIN, the single-user authentication delay is 24.8719 ms, including the computational overhead $11T_{pm} + 5T_{pa} + 9T_h$ and the transmission delay $2T_{s-g}$. When n users request access simultaneously, the total delay is $T_{batch} = (8n + 3)T_{pm} + 5nT_{pa} + (8n + 1)T_h + 2nT_{s-g} = 23.5458n + 1.3261$ ms. As shown in Table 2, PABV-STIN with

Table 1. Computational cost of cryptographic operations

Symbol	Description	Time (ms)
T_{pm}	Scalar multiplication of elliptic curve point E_p	0.4420
T_{pa}	Point addition operation on E_p	0.0018
T_h	Hash function operation	0.0001
T_{bp}	Bilinear pairing operation	4.2110
T_{exp}	Modular exponentiation operation	0.4467
T_{SE}	Symmetric encryption/decryption operation	0.0068
T_{end}	Asymmetric encryption/decryption operation	0.1303

Table 2. Comparison of batch authentication overhead and delay

Scheme	Batch Authentication Overhead	Transmission Delay	Total Delay (ms)
[38]	$17nT_{pm} + 7nT_{bp} + 3nT_{exp} + 2nT_h$	$2nT_{s-g}$	$58.3313n$
[31]	$11nT_{pm} + 7nT_{pa} + 35nT_h$	$2nT_{s-g} + nT_{g-g}$	$29.8781n$
[11]	$15nT_{pm} + 10nT_{pa} + 11nT_h$	$4nT_{s-g}$	$46.6491n$
[21]	$10nT_{pm} + 4nT_{pa} + 6nT_h + 2nT_{SE}$	$2nT_{s-g}$	$24.4414n$
[33]	$8nT_{pm} + 13nT_h + 4nT_{end}$	$4nT_{s-g}$	$44.0585n$
Our	$11nT_{pm} + 5nT_{pa} + 9nT_h$	$2nT_{s-g}$	$24.8719n$
Our batch	$(8n+3)T_{pm} + 5nT_{pa} + (8n+1)T_h$	$2nT_{s-g}$	$23.5458n + 1.3261$

batch verification achieves lower delay than the non-batch mode and most existing schemes. When $n = 1000$, it reduces the authentication delay by 59.63%, 21.19%, 49.52%, 3.66%, and 46.55% compared with protocols [38, 31, 11, 21, 33], respectively.

6.3 Backtracking Overhead

Let $m = \rho N$ be the number of invalid requests and $L = \lceil \log_2 N \rceil$. Since each invalid request induces at most one failed path of depth L , the number of sub-batch checks satisfies $V \leq \min\{2\rho NL, 2N - 2\}$. Let T_{BT} and T_{ind} denote the backtracking localization overhead and full individual re-verification overhead, respectively. With the cached prefix aggregates, each sub-batch check requires only one elliptic-curve point operation, yielding $T_{BT} \leq VT_{pa} \leq (2N - 2)T_{pa}$. By contrast, fully re-executing the individual cryptographic verification for all N original requests requires $T_{ind} = N(3T_{pm} + 2T_{pa} + 2T_h)$. For $N = 1000$, the conservative backtracking bound is 3.5964 ms, whereas full individual re-verification requires 1329.8 ms. Thus, although the backtracking overhead increases with ρ , it remains bounded and avoids repeating expensive scalar multiplications and hash computations.

Table 3. Comparison of communication overhead

Protocol	U_i (bits)	L_e (bits)	NCC(bits)	Total(bits)
[38]	1516	1624	–	3140
[31]	1152	832	672	2656
[11]	812	1996	672	3480
[21]	1252	2064	–	3316
[33]	964	2120	964	4048
Our	1224	2196	–	3420
Our Batch	$1224n$	$452n + 1904$	–	$1676n + 1904$

6.4 Communication Overhead Comparison

This subsection compares the communication overhead in the authentication phase. For fair comparison, the lengths of an identity, a timestamp, a hash value, and an encryption-related parameter are set to 100 bits, 32 bits, 160 bits, and 128 bits, respectively. For ECC-based values, the length of an elliptic-curve point is set to 320 bits, and the length of an element in $\mathbb{Z}_q^*$ is set to 160 bits. The user proxy credential and satellite proxy credential are set as $|w_{U_i}| = 292$ bits and $|w_{se}| = 392$ bits. As shown in Table 3, for PABV-STIN, U_i sends $\{M_{U_i}, s_{U_i}\}$, where $M_{U_i} = R_{U_i} \parallel ts_1 \parallel TID_{U_i} \parallel K_{U_i} \parallel w_{U_i}$. Thus, the user-side communication overhead is $C_{U_i} = 1224$ bits. The LEO satellite returns $\{M_{L_e}, s_{L_e}\}$, and the satellite-side communication overhead is $C_{L_e} = 2196$ bits. Therefore, the total single-user communication overhead is $C^{PABV} = 3420$ bits.

In the batch authentication scenario with n concurrent users, each user sends an independent authentication request, resulting in $C_U^{batch} = 1224n$ bits. The satellite broadcasts $\{List, M_{L_e}^{batch}, s_{L_e}^{batch}\}$, leading to $C_L^{batch} = 452n + 1904$ bits. Here, $452n$ comes from $List = \{TID_{U_i}, R_{U_i}, ts_1\}_{i=1}^n$, and 1904 bits come from the common fields in $M_{L_e}^{batch}$ and the batch response signature $s_{L_e}^{batch}$. Therefore, the total batch communication overhead is $C^{batch} = 1676n + 1904$ bits. When $n = 1000$, the communication overhead is 1,677,904 bits, which is reduced by 46.56%, 36.83%, 51.78%, 49.40%, and 58.55% compared with protocols [38, 31, 11, 21, 33], respectively.

As shown in Fig. 4, PABV-STIN-Batch shows lower authentication delay and communication overhead than the compared schemes under different numbers of users. Meanwhile, the reduction in delay and communication overhead becomes more obvious as the number of users increases. Therefore, PABV-STIN-Batch is more suitable for massive concurrent access scenarios.

7 Conclusion

This paper proposed PABV-STIN, a proxy-authorized batch verification protocol for massive IoT access in STIN. PABV-STIN enables authorized LEO satellites to authenticate users within a delegated scope by issuing time- and

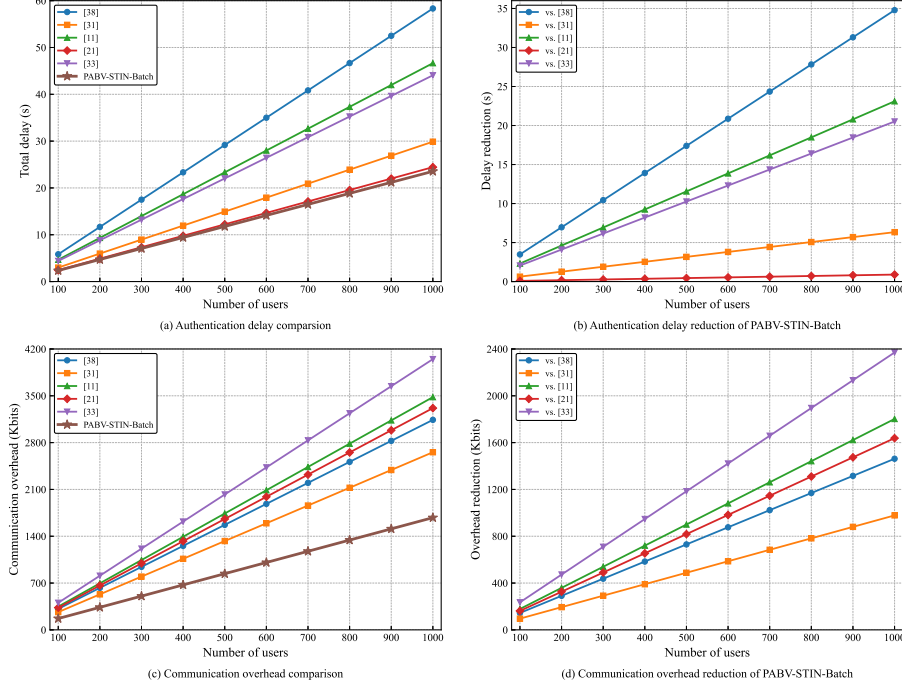


Fig. 4. Performance comparison under different numbers of users.

scope-constrained proxy credentials before user access authentication. It also uses random-coefficient-based batch verification and binary backtracking to reduce verification overhead and locate abnormal requests when batch verification fails. AVISPA-assisted verification, heuristic security analysis, and performance evaluation show that PABV-STIN satisfies the required security properties and achieves lower authentication delay and communication overhead than representative related protocols. Future work will study adaptive batch partitioning under high invalid-request ratios.

Disclosure of Interests The authors have no competing interests to declare.

Appendix

For ease of reference, the main notations used in PABV-STIN are summarized in Table 4.

Table 4. Main notations used in PABV-STIN

Notation	Description
G, q, P	Cyclic additive group, its order, and generator
x, P_{NCC}	Master private key and system public key of the NCC
$H(\cdot), h(\cdot)$	Secure hash functions
k	Security parameter used in batch verification
$K_s, K_e, K_{U_i}, K_{se}$	Public parameters generated from r_s, r_e, r_{U_i} , and r_{se}
ID_s, ID_e, ID_{U_i}	Identities of G_s, L_e , and U_i
TID_{U_i}	Temporary identity of user U_i
$r_s, r_e, r_{U_i}, r_{se}$	Random values used for credential generation
$\sigma_s, \sigma_e, \sigma_{U_i}$	Registration authentication credentials of G_s, L_e , and U_i
σ_{se}	Temporary proxy signing key of L_e
w_{U_i}, w_{se}	User access credential and satellite proxy warrant
n_s, n_e, b_i, c_{L_e}	Ephemeral random values used in authentication
$N_s, N_e, R_{U_i}, R_{L_e}$	Ephemeral elliptic-curve points generated from n_s, n_e, b_i , and c_{L_e}
$M_s, M_e, M_{se}, M_{U_i}, M_{L_e}$	Authentication and delegation messages used in different phases
$S_s, S_e, s_{U_i}, s_{L_e}$	Authentication signatures used in different phases
C_{se}, Tag_{se}	Ciphertext and message authentication code for proxy delegation
$ts_s, ts_e, ts_{se}, ts_1, ts_2$	Timestamps used in different authentication phases
SK_{A-B}	Session key shared between entities A and B
N, α_i	Number of users and random coefficient for batch verification
$List$	User-information list used in batch authentication
$M_{L_e}^{\text{batch}}, s_{L_e}^{\text{batch}}$	Batch response message and batch response signature
Q_i	Weighted verification residual of request U_i
S_j	Prefix aggregate used for abnormal-request localization

```

role leo(Gs,Le,Ui: agent,
Ks,Ke,Kui,TidUi,Wui,Dhul,P: text,
SigGs,SigLe,SigUi: text,
PKse: public_key,
Kgl: symmetric_key,SND,RCV: channel(dy),H1,H2: hash_func)
played_by Le
def=localState: nat,Ns, e,Tss,Tse,Tsse: text, Kse,Wse: text,Rui,Rle,Ts1,Ts2: text,Cle: text,SKlu: text
Init State := 0
transition
1. State = 0 ∧ RCV(Gs.Ks.Ns'.Tss'.H1(Gs.Ks.Ns'.Le.Tss'.SigGs))=>
  State' := 1
  ∧ request(Le,Gs,gs_le_auth,Ns') ∧ Ne' := new() ∧ Tse' := new()
  ∧ SND(Le.Ke.Ne'.Ns'.Tse'.H1(Le.Ke.Ne'.Gs.Tse'.SigLe))
  ∧ witness(Le,Gs,le_gs_auth,Ne')
2. State = 1
  ∧ RCV({Gs.Le.Ks.Kse'.PKse.Wse'}_Kgl.H1({Gs.Le.Ks.Kse'.PKse.Wse'}_Kgl.Tsse'.Kgl).Tsse') =>
  State' := 2 ∧ request(Le,Gs,gs_le_proxy,Wse')
3. State = 2 ∧ RCV(Rui'.Ts1'.TidUi.Kui.Wui.H1(Rui'.Ts1'.TidUi.Kui.Wui.SigUi)) =>
  State' := 3 ∧ request(Le,Ui,ui_le_auth,Rui') ∧ request(Le,Ui,ui_le_ts1,Ts1')
  ∧ Cle' := new() ∧ Rle' := H1(Cle'.P) ∧ Ts2' := new()
  ∧ SKlu' := H2(Dhul.Rui'.Rle'.TidUi.Le.Ts1'.Ts2') ∧ secret(SKlu',sk_ui_le,{Ui,Le}) ∧
SND(Rle'.Rui'.TidUi.Ts1'.Ts2'.Le.Gs.Kse.Ks.Wse,{Rle'.Rui'.TidUi.Ts1'.Ts2'.Le.Gs.Kse.Ks.Wse}_inv(PKse))
  ∧ witness(Le,Ui,le_ui_auth,Rle')
  ∧ witness(Le,Ui,le_ui_ts2,Ts2')
end role

```

(a) LEO Satellite Role

```

role gs( Gs,Le,Ui: agent,
Ks,Ke,Kui,TidUi,Wui,Dhul,P: text,
SigGs,SigLe,SigUi: text,
PKse: public_key,
Kgl: symmetric_key,
SND,RCV: channel(dy),H1,H2: hash_func)
played_by Gs
def=local State: nat,
Ns,Ne,Tss,Tse,Tsse: text,Kse,Wse: text, Cse,Tagse:
message
Init State := 0
transition
1. State = 0 ∧ RCV(start)=>
  State' := 1 ∧ Ns' := new() ∧ Tss' := new()
  ∧ SND(Gs.Ks.Ns'.Tss'.H1(Gs.Ks.Ns'.Le.Tss'.SigGs))
  ∧ witness(Gs,Le,gs_le_auth,Ns')
2. State = 1 ∧
RCV(Le.Ke.Ne'.Ns.Tse'.H1(Le.Ke.Ne'.Ns.Gs.Tse'.SigLe))
=>
  State' := 2
  ∧ request(Gs,Le,le_gs_auth,Ne') ∧
secret(Kgl,sk_gs_le,{Gs,Le})
  ∧ Kse' := new() ∧ Wse' := new() ∧ Tsse' := new()
  ∧ Cse' := {Gs.Le.Ks.Kse'.PKse.Wse'}_Kgl
  ∧ Tagse' := H1(Cse'.Tsse'.Kgl) ∧ SND(Cse'.Tagse'.Tsse')
  ∧ witness(Gs,Le,gs_le_proxy,Wse')
end role

```

(b) GS Role

```

role session( Ui,Le,Gs: agent,
Ks,Ke,Kui,TidUi,Wui,Dhul,P: text,
SigGs,SigLe,SigUi: text,
PKse: public_key,
Kgl: symmetric_key,H1,H2: hash_func)
def=local SND,RCV: channel(dy)
composition
gs(Gs,Le,Ui,Ks,Ke,Kui,TidUi,Wui,Dhul,P,SigGs,SigLe,SigUi,PKse,Kgl,SND,RCV,H1,H2)
  ∧
leo(Gs,Le,Ui,Ks,Ke,Kui,TidUi,Wui,Dhul,P,SigGs,SigLe,SigUi,PKse,Kgl,SND,RCV,H1,H2)
  ∧
user(Ui,Le,Gs,Ks,Ke,Kui,TidUi,Wui,Dhul,P,SigUi,PKse,SND,RCV,H1,H2)
end role
role environment() def=
const
  ui,le,gs,i: agent,ks,ke,kui,tidui,wui,dhul,p: text,siggs,sigle,sigui: text,
  pkse: public_key,kgl: symmetric_key,h1,h2: hash_func,
  gs_le_auth,le_gs_auth,gs_le_proxy,ui_le_auth,le_ui_auth,ui_le_ts1,
  le_ui_ts2,sk_gs_le,sk_ui_le: protocol_id
  intruder_knowledge = {ui,le,gs,i,ks,ke,kui,tidui,wui,p,pkse,h1,h2}
composition
session(ui,le,gs,ks,ke,kui,tidui,wui,dhul,p,siggs,sigle,sigui,pkse,kgl,h1,h2)
end role

```

(c) Session and Environment Roles

Fig. 5. HPSL specifications of the remaining roles.

References

1. Ahmad, I., Suomalainen, J., Porambage, P., Gurtov, A., Huusko, J., Höyhtyä, M.: Security of satellite-terrestrial communications: Challenges and potential solutions. *IEEE Access* **10**, 96038–96052 (2022). <https://doi.org/10.1109/ACCESS.2022.3205426>
2. Altaf, I., Akram, M.A., Mahmood, K., Kumari, S., Xiong, H., Khan, M.K.: A novel authentication and key-agreement scheme for satellite communication network. *Transactions on Emerging Telecommunications Technologies* **32**(7), e3894 (2021). <https://doi.org/10.1002/ett.3894>
3. Canetti, R., Krawczyk, H.: Analysis of key-exchange protocols and their use for building secure channels. In: Pfitzmann, B. (ed.) *Advances in Cryptology – EUROCRYPT 2001*. Lecture Notes in Computer Science, vol. 2045, pp. 453–474. Springer, Berlin, Heidelberg (2001). https://doi.org/10.1007/3-540-44987-6_28
4. Chen, Y., Chen, J.: An enhanced dynamic authentication scheme for mobile satellite communication systems. *International Journal of Satellite Communications and Networking* **39**(3), 250–262 (2021). <https://doi.org/10.1002/sat.1385>
5. Dolev, D., Yao, A.C.: On the security of public key protocols. *IEEE Transactions on Information Theory* **29**(2), 198–208 (1983). <https://doi.org/10.1109/TIT.1983.1056650>
6. Gautam, D., Kanwar, S., Prajapat, S., Kumar, P., Chen, C.M.: A robust ECC based authentication protocol for satellite-to-satellite communication network. *Telecommunication Systems* **87**, 541–559 (2024). <https://doi.org/10.1007/s11235-024-01199-2>
7. Guo, J., Du, Y.: A secure three-factor anonymous roaming authentication protocol using ECC for space information networks. *Peer-to-Peer Networking and Applications* **14**, 898–916 (2021). <https://doi.org/10.1007/s12083-020-01050-w>
8. Guo, J., Du, Y., Sun, Z., Wu, R., Wu, X., Zhang, L., Zheng, T.: SRAKN: Secure roaming authentication and key negotiation protocol for space information network. *Computer Communications* **206**, 22–37 (2023). <https://doi.org/10.1016/j.comcom.2023.04.011>
9. Guo, J., Du, Y., Wu, X., Li, M., Wu, R., Sun, Z.: PSAA: Provable secure and anti-quantum authentication based on randomized RLWE for space information network (2022), arXiv:2208.00901
10. Guo, J., Du, Y., Zhang, D., Wu, R.: PSEEMV: Provably secure and efficient emergency message verification scheme based on ECC and CRT for space information network. *Journal of Information Security and Applications* **73**, 103437 (2023). <https://doi.org/10.1016/j.jisa.2023.103437>
11. Guo, J., Du, Y., Zhang, Y., Li, M.: A provably secure ECC-based access and handover authentication protocol for space information networks. *Journal of Network and Computer Applications* **193**, 103183 (2021). <https://doi.org/10.1016/j.jnca.2021.103183>
12. Guo, J., Yao, S., Song, Y., Han, X., Zheng, Z., Chang, L.: N3PA-STIN: A novel three-party authentication protocol for multi-user access in satellite-terrestrial integrated networks. *IEEE Internet of Things Journal* **12**(10), 14952–14968 (2025). <https://doi.org/10.1109/JIOT.2025.3527214>
13. Huang, C., Zhang, Z., Li, M., Zhu, L., Zhu, Z., Yang, X.: A mutual authentication and key update protocol in satellite communication network. *Automatika* **61**(3), 334–344 (2020). <https://doi.org/10.1080/00051144.2020.1757966>

14. Kang, M., Park, S., Lee, Y.: A survey on satellite communication system security. *Sensors* **24**(9), 2897 (2024). <https://doi.org/10.3390/s24092897>
15. Kim, S., Park, S., Won, D.: Proxy signatures, revisited. In: Han, Y., Okamoto, T., Qing, S. (eds.) *Information and Communications Security. Lecture Notes in Computer Science*, vol. 1334, pp. 223–232. Springer, Berlin, Heidelberg (1997). <https://doi.org/10.1007/BFb0028478>
16. Koblitz, N.: Elliptic curve cryptosystems. *Mathematics of Computation* **48**(177), 203–209 (1987). <https://doi.org/10.1090/S0025-5718-1987-0866109-5>
17. Kumar, U., Garg, M.: Learning with error-based key agreement and authentication scheme for satellite communication. *International Journal of Satellite Communications and Networking* **40**(2), 83–95 (2022). <https://doi.org/10.1002/sat.1417>
18. Lakew, D.S., Tran, A.T., Masood, A., Dao, N.N., Cho, S.: A review on satellite-terrestrial integrated wireless networks: Challenges and open research issues. In: 2023 International Conference on Information Networking (ICOIN). pp. 638–641. IEEE (2023). <https://doi.org/10.1109/ICOIN56518.2023.10049009>
19. Li, F., Yin, L., Wu, W., Zhang, L., Shi, G.: Research status and development trends of security assurance for space-ground integration information network. *Journal on Communications* **37**(11), 156–168 (2016). <https://doi.org/10.11959/j.issn.1000-436x.2016229>, (in Chinese)
20. Li, J., Cao, Z., Li, J., Zhang, Y.: Present situation and progress of proxy signature. *Journal on Communications* **24**(10), 114–124 (2003), (in Chinese)
21. Liu, Y., Ni, L., Peng, M.: A secure and efficient authentication protocol for satellite-terrestrial networks. *IEEE Internet of Things Journal* **10**(7), 5810–5822 (2023). <https://doi.org/10.1109/JIOT.2022.3152900>
22. Maral, G., Bousquet, M., Sun, Z.: *Satellite Communications Systems: Systems, Techniques and Technology*. John Wiley & Sons, Hoboken, 6th edn. (2020). <https://doi.org/10.1002/9781119673811>
23. Mishra, D., Rewal, P., Pursharthi, K.: Efficient and quantum-secure authenticated key exchange scheme for mobile satellite communication networks. *International Journal of Satellite Communications and Networking* **42**(4), 313–328 (2024). <https://doi.org/10.1002/sat.1516>
24. Ramesh, J.V.N., Nimma, D., Dasari, R., AlGhamdi, A., Keshta, I., Tata, R.K., Soni, M., Kiyosov, S., Jangir, P.: Satellite IoT terminal authentication mechanism for enhancing consumer security over cyber-physical systems. *IEEE Transactions on Consumer Electronics* **71**(2), 4771–4779 (2025). <https://doi.org/10.1109/TCE.2025.3535078>
25. Rivest, R.L., Shamir, A., Adleman, L.: A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM* **21**(2), 120–126 (1978). <https://doi.org/10.1145/359340.359342>
26. Schnorr, C.P.: Efficient identification and signatures for smart cards. In: Brassard, G. (ed.) *Advances in Cryptology – CRYPTO ’89. Lecture Notes in Computer Science*, vol. 435, pp. 239–252. Springer, New York (1990). https://doi.org/10.1007/0-387-34805-0_22
27. Schnorr, C.P.: Efficient signature generation by smart cards. *Journal of Cryptology* **4**(3), 161–174 (1991). <https://doi.org/10.1007/BF00196725>
28. Shen, Y., Zhou, Q., Wen, Y., Shuai, Z., Shen, Z.J.: Integrated satellite-terrestrial network framework for next generation smart grid. *IEEE Transactions on Smart Grid* **15**(5), 5249–5252 (2024). <https://doi.org/10.1109/TSG.2024.3424150>
29. Soltani, S., Hosseini Seno, S.A., Rejito, J., Budiarto, R.: Robust authentication and session key agreement protocol for satellite communications. *Computers, Materials & Continua* **71**(3), 5891–5910 (2022). <https://doi.org/10.32604/cmc.2022.023697>

30. Song, J., Liu, T., Chen, X., Wu, Z.: Satellite navigation message authentication in GNSS: Research on message scheduler for SBAS L1. *Sensors* **24**(2), 360 (2024). <https://doi.org/10.3390/s24020360>
31. Srinivas, J., Das, A.K., Wazid, M., Vasilakos, A.V.: Designing secure user authentication protocol for big data collection in IoT-based intelligent transportation system. *IEEE Internet of Things Journal* **8**(9), 7727–7744 (2021). <https://doi.org/10.1109/JIOT.2020.3040938>
32. Tao, Y., Du, H., Xu, J., Su, L., Cui, B.: On-demand anonymous access and roaming authentication protocols for 6G satellite-ground integrated networks. *Sensors* **23**(11), 5075 (2023). <https://doi.org/10.3390/s23115075>
33. Thakur, G., Kumar, P., Deepika, Jangirala, S., Das, A.K., Park, Y.: An effective privacy-preserving blockchain-assisted security protocol for cloud-based digital twin environment. *IEEE Access* **11**, 26877–26892 (2023). <https://doi.org/10.1109/ACCESS.2023.3249116>
34. Wu, M., Wu, W., Zhou, B., Lu, Z., Zhang, P., Qin, Z.: General architecture consideration of space-ground integrated information network system. *Satellite & Network* (3), 30–36 (2016), (in Chinese)
35. Wu, Z., Zhang, Y., Yang, Y., Wang, P., Meng, Y.: BDSec: Security authentication protocol for BeiDou-II civil navigation message. *China Communications* **21**(6), 206–218 (2024). <https://doi.org/10.23919/JCC.ja.2022-0368>
36. Xu, L., Wu, H., Xie, J., Yuan, Q., Sun, Y., Shi, G., Luo, S.: An SSL-PUF based access authentication and key distribution scheme for the space-air-ground integrated network. *Entropy* **25**(5), 760 (2023). <https://doi.org/10.3390/e25050760>
37. Xu, Z., Peng, H., Gu, K., Li, X., Huang, P.: An energy efficient access and handover authentication scheme for 6G satellite-terrestrial integrated network. *IEEE Transactions on Green Communications and Networking* **9**(2), 684–697 (2025). <https://doi.org/10.1109/TGCN.2024.3450515>
38. Yang, Q., Xue, K., Xu, J., Wang, J., Li, F., Yu, N.: AnFRA: Anonymous and fast roaming authentication for space information network. *IEEE Transactions on Information Forensics and Security* **14**(2), 486–497 (2019). <https://doi.org/10.1109/TIFS.2018.2854740>
39. Yin, J., Chen, Y., Lin, X., Zhao, Q.: The remote sensing data transmission problem in communication constellations: Shop scheduling-based model and algorithm. *Technologies* **13**(9), 384 (2025). <https://doi.org/10.3390/technologies13090384>
40. Zhou, Z., Ma, J., Wan, G., Su, L., Du, H., Cui, B.: Research on 6G satellite-terrestrial integrated network architecture. In: 7th International Conference on Cyber Security and Information Engineering (ICCSIE 2022). pp. 1057–1062. ACM, New York, NY, USA (2022). <https://doi.org/10.1145/3558819.3565467>